Reading IMU Data

I. Experiment Objective

Learn how to read the raw data of the QMI8658 six-axis inertial sensor (accelerometer + gyroscope) through the I2C bus on the ESP32-S3 platform, and solve it into Euler angles (roll, pitch, yaw). Also master the basic methods of sensor calibration, attitude fusion algorithms (AHRS), and attitude stability analysis.

II. Experiment Principle

1. Keywords

Keyword	Explanation
IMU (Inertial Measurement Unit)	A sensor module that integrates an accelerometer and a gyroscope. The accelerometer measures three-axis acceleration (including the gravity component), and the gyroscope measures three-axis angular velocity. The QMI8658 is a six-axis IMU that does not include a magnetometer.
Accelerometer	Measures the acceleration on three axes (unit m/s^2). At rest, it detects the direction of gravity (vertically downward) and is used to determine the pitch angle and roll angle. In this experiment, the default range is ±8g and the sensitivity is 1<<13.
Gyroscope	Measures the angular velocity on three axes (unit rad/s). Integration yields the angle change. In this experiment, the default range is ±1024 dps and the sensitivity is 32. At startup, the zero offset must be calibrated to reduce temperature drift.
AHRS (Attitude and Heading Reference System)	An attitude solving algorithm that fuses accelerometer and gyroscope data. This experiment uses the open-source Fusion library, which performs six-axis fusion (no magnetometer), outputs quaternions, and converts them to Euler angles. Uses the North-West-Up (NWU) coordinate convention.
Quaternion / Euler angle	A quaternion (q0,q1,q2,q3) is a mathematical representation of attitude with no gimbal-lock problem. Euler angles (roll, pitch, yaw) convert the quaternion into intuitive angle values. Since there is no magnetometer, yaw drifts over time.

2. Technical Architecture

The program adopts a layered driver structure: I2C master driver → QMI8658 sensor driver (platform adaptation layer) → attitude fusion encapsulation layer (Fusion AHRS library) → attitude stability analyzer.

```
main/main.c
    └── imu_attitude_fusion_init() / update_imu_data() / imu_attitude_fusion_update()
         └── components/imu_attitude_fusion/     Attitude fusion encapsulation layer
              ├── components/qmi8658b/          QMI8658 sensor driver (register read/write,
data conversion)
              |    └── components/qmi8658b_port_esp32/  ESP32 platform adaptation (binding
I2C callbacks)
              |         └── components/i2c_master/      I2C master driver (SCL:GPIO39,
SDA:GPIO40)
              └── components/Fusion/                   Third-party AHRS algorithm library
         └── components/attitude_stability_analyzer/    Attitude stability analyzer (window
statistics)
```

The main loop runs every 10ms: read sensor data → subtract the gyroscope zero offset → perform AHRS fusion → get Euler angles → perform a stability analysis once per second.

III. Experiment Steps

1. Hardware Preparation

Hardware	Requirement
MicroROS-V2 control board	✅ Required
USB data cable (Type-C)	✅ Required

2. Prerequisites

1. Set up the ESP32-IDF development environment
2. Connect the MicroROS-V2 control board to the computer via USB
3. Place the control board horizontally and still so that the gyroscope can be calibrated at startup

3. Compile and Flash

```
cd ~/esp/base_samples/read_imu
source ~/esp/esp-idf/export.sh
idf.py build flash monitor
```

To exit the monitor: press `Ctrl + ]`.

4. Stopping Method

In the terminal, press `Ctrl+]` to exit the serial monitor.

IV. Experiment Result

After flashing and running the program, you can see the following output in the serial monitor terminal:

```
hello yahboom
I (1002) IMU_MAIN: Nice to meet you!
I (1012) I2C_MASTER: Init I2C master:SCL->GPIO39, SDA->GPIO40
qmi8658 slave = 0x6b WhoAmI = 0x05
...
I (6012) IMU_MAIN: Calibration complete, starting stability analysis
I (6012) IMU_MAIN: First ok Euler angles: roll = 0.52, pitch = -0.83, yaw = 0.02
I (6022) IMU_MAIN: Euler angles: roll = 0.53, pitch = -0.84, yaw = 0.05
I (6032) IMU_MAIN: Euler angles: roll = 0.51, pitch = -0.82, yaw = -0.01
```

- The first 5 seconds are the calibration phase, during which no Euler angles are output
- After calibration is complete, roll (roll angle), pitch (pitch angle), and yaw (yaw angle) are output every 10ms
- When the control board is horizontal and still, roll and pitch fluctuate around 0° (usually within ±1°), and yaw represents the current heading angle, which is close to 0° by default after a reset/reboot
- Slowly tilt or rotate the control board, and the Euler angles will change accordingly

V. Program Analysis

> Source code paths:
>
> - Firmware source: `~/esp/base_samples/read_imu/main/main.c`
> - Attitude fusion: `~/esp/base_samples/read_imu/components/imu_attitude_fusion/imu_attitude_fusion.h`, `imu_attitude_fusion.c`
> - I2C driver: `~/esp/base_samples/read_imu/components/i2c_master/i2c_master.h`, `i2c_master.c`
> - QMI8658 driver: `~/esp/base_samples/read_imu/components/qmi8658b/qmi8658b.h`, `qmi8658b.c`

1. Directory Structure

```
read_imu/
├── CMakeLists.txt
├── main/
|   ├── CMakeLists.txt
|   └── main.c
└── components/
    ├── qmi8658b/                    # QMI8658 sensor driver
    |   ├── qmi8658b.h
    |   └── qmi8658b.c
    ├── qmi8658b_port_esp32/         # ESP32 platform adaptation layer
    |   ├── qmi8658b_port_esp32.h
    |   └── qmi8658b_port_esp32.c
    ├── i2c_master/                  # I2C master driver
    |   ├── i2c_master.h
    |   └── i2c_master.c
    ├── imu_attitude_fusion/         # Attitude fusion encapsulation layer
```

```
    |   ├── imu_attitude_fusion.h
    |   └── imu_attitude_fusion.c
    ├── Fusion/                        # Third-party AHRS fusion algorithm library
    |   ├── Fusion.h
    |   ├── FusionAhrs.h / .c
    |   ├── FusionBias.h / .c
    |   ├── FusionMath.h
    |   ├── FusionEuler.h
    |   └── ...
    └── attitude_stability_analyzer/   # Attitude stability analyzer
        ├── attitude_stability_analyzer.h
        └── attitude_stability_analyzer.c
```

2. Main Program Logic

`main/main.c` implements the IMU data reading and attitude solving flow:

1. Call `imu_attitude_fusion_init()` to initialize the I2C, the QMI8658 sensor, and the Fusion algorithm library
2. Call `fusion_param_setting()` to set the sample rate (100Hz) and the gyroscope range (1024 dps)
3. Call `calibrate_gyro_startup_bias()` to perform gyroscope startup bias calibration (sampling 300 times to calculate the average)
4. Initialize the attitude stability analyzer (window size 300 samples)
5. Main loop:
    - Wait for a 5-second calibration time; `calibrate_time_counter` increments until it reaches `CALIBRATION_WAIT_TIME_MS（5000ms）/ 10 = 500` times
    - After calibration, set `calibrate_ok_flag` to 1 and start displaying Euler angles
    - Read sensor data every 10ms and update the AHRS fusion
    - Perform a stability analysis calculation once per second

Key configuration constants:

```
#define FUSION_SAMPLE_RATE_HZ     (100.0f)   // Sample rate 100Hz
#define FUSION_GYRO_RANGE_DPS     (1024.0f)  // Gyroscope range ±1024 dps
#define ANALYZER_WINDOW_SIZE      (300U)     // Stability analysis window of 300 samples
#define CALIBRATION_WAIT_TIME_MS  (5000U)    // Calibration wait time 5 seconds
```

3. Key Components

I2C master driver

Files: `components/i2c_master/i2c_master.h`, `components/i2c_master/i2c_master.c`

Pin configuration:

> Parameter definition file: `~/esp/base_samples/read_imu/components/i2c_master/i2c_master.h`

Signal	GPIO	Description
SCL	GPIO39	I2C clock line
SDA	GPIO40	I2C data line

Parameter configuration:

- I2C master number: 0 (`I2C_MASTER_NUM = 0`)
- Communication frequency: 400 KHz (standard fast mode)
- Timeout: 1000ms

Core functions:

- `I2C_Master_Init()` — Initializes the I2C master, configures the SCL/SDA pins, and installs the driver
- `I2C_Master_Read(uint8_t addr, uint8_t reg, uint16_t len, uint8_t* data)` — Reads multi-byte data from the register of the specified device address
- `I2C_Master_Write_Byte(uint8_t addr, uint8_t reg, uint8_t data)` — Writes a single byte of data to the register of the specified device address

QMI8658 sensor driver

Files: `components/qmi8658b/qmi8658b.h`, `components/qmi8658b/qmi8658b.c`

The QMI8658 is a six-axis inertial measurement unit (IMU) that integrates a three-axis accelerometer and a three-axis gyroscope.

Device information:

- I2C slave device address: 0x6b (ADDR_L) or 0x6a (ADDR_H)
- WhoAmI register value: 0x05
- Reset register: `Qmi8658Register_Reset = 96`, writing 0xB0 triggers a soft reset

Register map (partial, key registers):

Register	Address	Description
WhoAmI	0x00	Chip ID (should be 0x05)
Ctrl1	0x02	Interrupt and clock configuration
Ctrl2	0x03	Accelerometer configuration (range + ODR)
Ctrl3	0x04	Gyroscope configuration (range + ODR)
Ctrl7	0x08	Sensor enable control
Ax_L ~ Az_H	0x35~0x3A	Accelerometer raw data (6 bytes)
Gx_L ~ Gz_H	0x3B~0x40	Gyroscope raw data (6 bytes)
Status0	0x2E	Data ready status

Accelerometer ranges:

Enum Value	Range	Sensitivity
`Qmi8658AccRange_2g`	±2g	1<<14
`Qmi8658AccRange_4g`	±4g	1<<13
`Qmi8658AccRange_8g`	±8g	1<<12 (default)
`Qmi8658AccRange_16g`	±16g	1<<11

Gyroscope ranges:

Enum Value	Range	Sensitivity
`Qmi8658GyrRange_16dps`	±16 dps	2048
`Qmi8658GyrRange_32dps`	±32 dps	1024
`Qmi8658GyrRange_64dps`	±64 dps	512
`Qmi8658GyrRange_128dps`	±128 dps	256
`Qmi8658GyrRange_256dps`	±256 dps	128
`Qmi8658GyrRange_512dps`	±512 dps	64
`Qmi8658GyrRange_1024dps`	±1024 dps	32 (default)
`Qmi8658GyrRange_2048dps`	±2048 dps	16

Default configuration:

- Accelerometer: ±8g, 250Hz ODR
- Gyroscope: ±1024 dps, 250Hz ODR
- Synchronous sampling mode (`QMI8658_SYNC_SAMPLE_MODE`)

Core functions:

- `qmi8658_init()` — Initializes the sensor, detects the chip ID, performs a soft reset, performs one-time calibration, and configures registers
- `qmi8658_read_xyz(float acc[3], float gyro[3])` — Reads accelerometer (m/s²) and gyroscope (rad/s) data
- `qmi8658_soft_reset()` — Soft-resets the sensor
- `qmi8658_axis_convert(float data_a[3], float data_g[3], int layout)` — Axis direction remapping (supports 24 layouts)

Data reading flow:

Internal implementation of `qmi8658_read_sensor_data()` :

1. Read 12 bytes consecutively starting from register `Ax_L` (0x35)

2. Extract 6 signed 16-bit raw values (acc_x/y/z, gyro_x/y/z)
3. Convert the raw values to physical units:
    - Accelerometer: `acc = raw * 9.807 / ssvt_a` (unit m/s²)
    - Gyroscope: `gyro = raw * π / (ssvt_g * 180)` (unit rad/s)
4. Perform axis remapping (layout = 0, i.e., X=X, Y=Y, Z=Z)
5. If static calibration is enabled, subtract the zero offset

ESP32 platform adaptation layer

Files: `components/qmi8658b_port_esp32/qmi8658b_port_esp32.h`, `components/qmi8658b_port_esp32/qmi8658b_port_esp32.c`

Binds the bus interface callbacks of the QMI8658 driver to the ESP32 I2C driver:

```
qmi8658_bus_io_t bus_io = {
    .write_reg = qmi8658_esp32_write_reg,   // Calls I2C_Master_Write_Byte
    .read_reg  = qmi8658_esp32_read_reg,    // Calls I2C_Master_Read
    .delay_ms  = qmi8658_esp32_delay_ms,    // Calls vTaskDelay
    .delay_us  = qmi8658_esp32_delay_us,    // Calls esp_rom_delay_us
};
```

Attitude fusion layer

Files: `components/imu_attitude_fusion/imu_attitude_fusion.h`, `components/imu_attitude_fusion/imu_attitude_fusion.c`

Based on the open-source Fusion AHRS algorithm library, it implements six-axis attitude solving without a magnetometer (pitch, roll, yaw).

Initialization flow:

```
uint8_t imu_attitude_fusion_init(FusionAhrs* fusion, FusionBias* bias)
{
    I2C_Master_Init();
    ESP_ERROR_CHECK(qmi8658_esp32_port_register());
    if (!qmi8658_init()) return 0;
    FusionBiasInitialise(bias);
    FusionAhrsInitialise(fusion);
    return 1;
}
```

Algorithm parameter setting:

```
void fusion_param_setting(FusionAhrs* fusion, FusionBias* bias,
                          float sample_rate_hz, float gyro_range_dps)
{
    FusionBiasSettings bias_settings = {
        .sampleRate = sample_rate_hz,           // 100 Hz
        .stationaryThreshold = 3.0f,            // Stationary threshold
        .stationaryPeriod = 3.0f,               // Stationary determination period (seconds)
```

```
    };
    FusionBiasSetSettings(bias, &bias_settings);

    FusionAhrsSettings ahrs_settings = {
        .convention = FusionConventionNwu,      // North-West-Up coordinate system
        .gain = 0.5f,                           // Fusion gain
        .gyroscopeRange = gyro_range_dps,       // 1024 dps
        .accelerationRejection = 10.0f,         // Acceleration disturbance rejection
        .magneticRejection = 0.0f,              // No magnetometer
        .recoveryTriggerPeriod = 5.0f * sample_rate_hz,
    };
    FusionAhrsSetSettings(fusion, &ahrs_settings);
}
```

Gyroscope startup bias calibration:

`calibrate_gyro_startup_bias()` collects 300 gyroscope samples at startup and averages them as the static zero offset. Each subsequent call to `update_imu_data()` automatically subtracts this offset to reduce the effect of gyroscope temperature drift.

Core functions:

- `update_imu_data(FusionVector *acc, FusionVector *gyro)` — Reads sensor data, converts it to g and dps units, and subtracts the startup offset
- `imu_attitude_fusion_update()` — Performs AHRS fusion, inputs the corrected gyroscope and accelerometer data, and updates the attitude
- `imu_attitude_fusion_get_quaternion()` — Gets the current quaternion
- `imu_attitude_fusion_get_euler()` — Converts the quaternion to Euler angles (roll, pitch, yaw, in degrees)

Attitude stability analyzer

Files: `components/attitude_stability_analyzer/attitude_stability_analyzer.h`, `components/attitude_stability_analyzer/attitude_stability_analyzer.c`

Used to evaluate the stability of the attitude angles over a period of time, calculating statistical metrics such as mean, standard deviation, min/max, and drift rate.

Data structure:

```
typedef struct {
    float roll_buffer[300];      // Roll history ring buffer
    float pitch_buffer[300];     // Pitch history data
    float yaw_buffer[300];       // Yaw history data
    float time_buffer[300];      // Timestamp history
    size_t window_size;          // Window size (max 300)
    size_t count;                // Current sample count
    size_t head;                 // Ring buffer head pointer
} attitude_stability_analyzer_t;
```

Output metrics:

```c
typedef struct {
    axis_stability_metrics_t roll;    // Roll statistics
    axis_stability_metrics_t pitch;   // Pitch statistics
    axis_stability_metrics_t yaw;     // Yaw statistics
    size_t sample_count;
    float window_duration_s;          // Window duration (seconds)
    bool valid;
} attitude_stability_result_t;
```

The statistical metrics for each axis include: mean, stddev (standard deviation), min, max, range, drift_rate_deg_per_sec (drift rate per second), drift_rate_deg_per_min (drift rate per minute), and cumulative_drift_deg (cumulative drift).

VI. Common Problems

Problem	Cause	Solution
No IMU data	Abnormal I2C connection or sensor initialization failure	Check whether GPIO39/40 are loose and confirm that the sensor WhoAmI is 0x05
roll/pitch is not 0	The control board is not placed horizontally or the sensor installation has deviation	Place the control board on a horizontal desk, or modify the axis mapping layout
Large Euler angle fluctuation	Acceleration disturbance or inappropriate algorithm gain	Check the `gain` parameter (default 0.5) and reduce it appropriately
yaw drifts continuously	The gyroscope zero offset is not fully eliminated	Increase the calibration sample count (`sample_count` default 300) or extend the calibration wait time